Aula 19 — Correção da prova

> **Unidade 02: explicar o resultado e o raciocínio**

Da variante A à diferença na B.

IA Nível 0: faça as previsões e justificativas sem IA.

COMO VAMOS PERCORRER A PROVA

Do resultado à habilidade

1. Leia o trecho da **A** e formule uma previsão.
2. Confira o caminho que determina o resultado.
3. Identifique a habilidade avaliada.
4. Veja o que muda na **B**.

Cada questão reinicia o cenário. Alterações locais valem só nela.

Um objeto pode oferecer várias capacidades

```
public interface Item { String descrever(); }
public interface Usavel { boolean usar(Personagem alvo); }
public interface Recarregavel {
    void recarregar(int quantidade);
    int getCargas();
}
```

`Equipamento` implementa os três contratos.
`Pocao` implementa `Item` e `Usavel`.

A guarda vem antes da alteração

```
public void receberDano(int quantidade) {
    if (quantidade > 0) {
        vida -= quantidade;
        if (vida < 0) { vida = 0; }
    }
}
```

O construtor guarda a vida inicial recebida.
Quantidade não positiva não altera a vida.

Recuperar vida também tem guarda e teto

```
public void recuperarVida(int quantidade) {
    if (quantidade > 0) {
        vida += quantidade;
        if (vida > 100) { vida = 100; }
    }
}
```

Chegar a zero não impede uma recuperação posterior neste código.

Q1 · VARIANTE A

Qual é a saída?

```
Personagem p = new Personagem(88);
p.recuperarVida(19);
p.receberDano(-8);
System.out.println(p.getVida());
p.receberDano(130);
p.recuperarVida(7);
p.recuperarVida(-5);
System.out.println(p.getVida());
```

Cada chamada recebe o estado anterior

Saídas: `100` **e** `7`**.**

`88 → 100 → 100 → 0 → 7 → 7`

Os negativos são ignorados. O teto e o piso valem a cada operação.

Habilidade: rastrear estado aplicando guardas e limites.

B: `86 → 100 → 100 → 0 → 9 → 9`. Saídas: `100` e `9`.

Recarregar soma antes de limitar

```
public void recarregar(int quantidade) {
    if (quantidade <= 0) { return; }
    cargas += quantidade;
    if (cargas > maximo) { cargas = maximo; }
}
```

Equipamento	Capacidade	Variação de vida
Cajado	3	−12
Amuleto	2	+15

O uso verifica carga, aplica e consome

```
public boolean usar(Personagem alvo) {
    if (cargas == 0) { return false; }
    aplicarEm(alvo);
    cargas--;
    return true;
}
```

`aplicarEm` consulta `getVariacaoVida()` no próprio objeto: variação negativa causa dano; as demais pedem recuperação.

Q2 · VARIANTE A

Qual é a saída?

```
Equipamento item = new Amuleto(0);
item.recarregar(5);
item.usar(new Personagem(61));
System.out.println(item.descrever());
```

Q2 · RESOLUÇÃO E HABILIDADE

O teto vem antes do consumo

Saída: `Amuleto: 1/2`.

```
0 + 5 → teto 2 → uso → saldo 1
```

O texto reúne nome, saldo atual e capacidade máxima.

Habilidade: combinar recarga, saturação e consumo.

B: recarga de quatro também satura em dois. Mesma saída.

A poção aceita uma única utilização

```
// Campo: private boolean consumida = false;
public boolean usar(Personagem alvo) {
    if (consumida) { return false; }
    alvo.recuperarVida(20);
    consumida = true;
    return true;
}
```

`descrever()` devolve `"Poção cheia"` ou `"Poção vazia"`, conforme o campo.

O inventário guarda e consulta os itens

```
// Campo inicializado com new ArrayList<>():
private List<Item> itens;
public void adicionar(Item item) { itens.add(item); }
public Item obter(int indice) { return itens.get(indice); }
public void listar() {
    for (Item item : itens) {
        System.out.println(item.descrever());
    }
}
```

Q3 · VARIANTE A

Qual é a saída?

```
Inventario inventario = new Inventario();
Pocao pocao = new Pocao();
inventario.adicionar(new Cajado(2));
inventario.adicionar(pocao);
pocao.usar(new Personagem(36));
inventario.listar();
```

A lista consulta os mesmos objetos

Saídas:

```
Cajado: 2/3
Poção vazia
```

A ordem é a de inclusão. A poção já foi consumida quando é descrita.

Habilidade: reconhecer estado atualizado via referência em coleção.

B: o cajado começa com uma carga: `Cajado: 1/3`; `Poção vazia`.

ATIVIDADE

Q4 · VARIANTE A

Qual é a saída?

```
Cajado cajado = new Cajado(1);
Inventario inventario = new Inventario();
inventario.adicionar(cajado);
inventario.adicionar(cajado);
Personagem alvo = new Personagem(40);
Usavel primeiro = (Usavel) inventario.obter(0);
Usavel segundo = (Usavel) inventario.obter(1);
primeiro.usar(alvo);
System.out.println(segundo.usar(alvo));
System.out.println(alvo.getVida());
```

Duas posições, um único cajado

Saídas: `false` e `28`.

Momento	Cargas	Vida do alvo
Antes dos usos	1	40
Primeiro uso	0	28
Segundo uso: recusa	0	28

Habilidade: reconhecer identidade e estado compartilhados.

B: vida inicial `41`. Saídas: `false` e `29`.

Equipar guarda; usar delega

```
// Campo inicialmente null:
private Usavel equipado;
public void equipar(Usavel item) { equipado = item; }
public boolean usarEquipadoEm(Personagem alvo) {
    if (equipado == null) { return false; }
    return equipado.usar(alvo);
}
```

O campo guarda a referência recebida na chamada a `equipar`.

Q5 · VARIANTE A

Qual é a saída?

```
Personagem usuario = new Personagem(74);
Personagem alvo = new Personagem(50);
Usavel escolha = new Cajado(1);
usuario.equipar(escolha);
escolha = new Amuleto(2);
usuario.usarEquipadoEm(alvo);
System.out.println(alvo.getVida());
```

Q5 · RESOLUÇÃO E HABILIDADE

A variável muda; o campo conserva a referência

Saída: `38`. O cajado causa doze de dano ao alvo de vida cinquenta.

Referência	Depois de equipar	Depois de reatribuir
`escolha`	cajado	amuleto
`usuario.equipado`	cajado	cajado

Habilidade: distinguir reatribuição local e referência em campo.

B: o alvo começa com `60`. Saída: `48`.

Q6 · VARIANTE A

Qual é a saída?

```
Personagem a = new Personagem(32);
Personagem b = new Personagem(68);
a.equipar(new Cajado(1));
b.equipar(new Amuleto(1));
a.usarEquipadoEm(b);
b.usarEquipadoEm(a);
System.out.println(a.getVida());
System.out.println(b.getVida());
```

O argumento identifica o alvo do efeito

Saídas: `47` **e** `56`**.**

Chamada	Objeto alterado	Vida final
`a.usarEquipadoEm(b)`	`b`	`68 − 12 = 56`
`b.usarEquipadoEm(a)`	`a`	`32 + 15 = 47`

Habilidade: acompanhar quem solicita e quem recebe o efeito.

B: vidas iniciais `34` e `66`. Saídas: `49` e `54`.

Q7 · VARIANTE A

Qual é a saída?

```
Item item = new Cajado(1);
Usavel uso = (Usavel) item;
Recarregavel recarga = (Recarregavel) item;
System.out.println(uso == recarga);
```

O cast não cria uma instância

Saída: `true`.

`item`, `uso` e `recarga` alcançam o mesmo cajado.

O cast permite tratar a referência por outro contrato compatível.
`==` compara a identidade alcançada pelas referências.

Habilidade: explicar identidade através de casts compatíveis.

B: trecho e resultado iguais: `true`.

ATIVIDADE

Q8 · VARIANTE A

Qual é a saída?

```
Personagem alvo = new Personagem(100);
Amuleto item = new Amuleto(1);
System.out.println(item.usar(alvo));
System.out.println(item.getCargas());
```

Q8 · RESOLUÇÃO E HABILIDADE

Aceitar o uso não exige aumentar a vida

Saídas: `true` e `0`.

Há uma carga: o uso é aceito, tenta recuperar vida e consome a carga.
O personagem já está no teto, então sua vida continua `100`.

Habilidade: distinguir aceitação, efeito observado e consumo.

B: mesmo trecho, mesmas saídas: `true` e `0`.

Q9 · VARIANTE A

Qual é a saída?

```
Personagem p = new Personagem(48);
System.out.println(p.usarEquipadoEm(p));
p.equipar(new Pocao());
if (p.usarEquipadoEm(p)) {
    p.receberDano(6);
}
if (p.usarEquipadoEm(p)) {
    p.receberDano(6);
}
System.out.println(p.getVida());
```

O retorno também controla o if

Saídas: `false` e `62`.

1. Sem equipado: recusa, sem mudar a vida `48`.
2. Primeiro uso da poção: `48 + 20 − 6 = 62`.
3. Segundo uso: recusa; o corpo do segundo `if` não executa.

Habilidade: seguir delegação, retorno e fluxo condicional.

B: começa com `49`; saídas `false` e `63`.

Q10 · CLASSE LOCAL DESTA QUESTÃO

Agora a subclasse troca o método completo

```
public class CajadoLivre extends Cajado {
    public CajadoLivre() {
        super(0);
    }

    @Override
    public boolean usar(Personagem alvo) {
        alvo.receberDano(5);
        return true;
    }
}
```

Q10 · VARIANTE A

Qual é a saída?

```
Equipamento item = new CajadoLivre();
Personagem alvo = new Personagem(53);
item.usar(alvo);
System.out.println(alvo.getVida());
```

Q10 · RESOLUÇÃO E HABILIDADE

A sobrescrita substitui a sequência herdada

Saída: `48`, pois `53 − 5 = 48`.

O objeto é `CajadoLivre`: executa seu próprio `usar`.
Não há chamada automática ao corpo de `Equipamento.usar`.

A guarda de cargas da base não é executada; a carga zero não bloqueia esse uso.

Habilidade: prever o despacho para uma sobrescrita completa.

B: alvo começa com `43`; saída `38`.

Q11 · ESTADO INICIAL

Preparar os itens não recarrega o amuleto

```
Inventario inventario = new Inventario();
Amuleto amuleto = new Amuleto(0);
inventario.adicionar(new Pocao());
inventario.adicionar(amuleto);
inventario.adicionar(new Cajado(1));
```

A poção não oferece `Recarregavel`; os dois equipamentos oferecem.

Q11 · VARIANTE A

Qual é a saída?

```
int recarregados = 0;
for (int i = 0; i < 3; i++) {
    Item item = inventario.obter(i);
    if (item instanceof Recarregavel) {
        Recarregavel r = (Recarregavel) item;
        r.recarregar(1);
        recarregados++;
    }
}
System.out.println(recarregados);
System.out.println(amuleto.getCargas());
```

Q11 · RESOLUÇÃO E HABILIDADE

A capacidade pertence ao objeto

Saídas: `2` e `1`.

Poção: não recarrega. Amuleto e cajado: recarregam.

O cast não copia o amuleto. A variável `amuleto` consulta a carga alterada pelo laço.

Habilidade: reconhecer capacidades e rastrear estado compartilhado.

B: mesmo trecho; saídas `2` e `1`.

A cura depende do sucesso da base

```
// UsavelComCura implements Usavel
private Usavel base; // recebido no construtor
public boolean usar(Personagem alvo) {
    if (!base.usar(alvo)) { return false; }
    alvo.recuperarVida(5);
    return true;
}
```

Cada camada chama sua base antes de acrescentar o efeito.

ATIVIDADE

Q12 · VARIANTE A

Qual é a saída?

```
Personagem alvo = new Personagem(98);
Usavel uso = new UsavelComCura(
    new UsavelComCura(new Cajado(1)));
uso.usar(alvo);
System.out.println(alvo.getVida());
uso.usar(alvo);
System.out.println(alvo.getVida());
```

Q12 · RESOLUÇÃO E HABILIDADE

A base age; as camadas curam na volta

Saídas: `96` **e** `96`**.**

`98 → cajado: 86 → camada interna: 91 → camada externa: 96`

No segundo uso, o cajado sem carga devolve `false`.
As duas camadas retornam antes da cura.

Habilidade: acompanhar composição e propagação de recusa.

B: `97 → 85 → 90 → 95`; saídas `95` e `95`.

Q13 · VARIANTE A

Por que o compilador rejeita descrever?

```
Usavel escolha = new Amuleto(1);
System.out.println(escolha.descrever());
```

A) Métodos herdados não podem ser chamados.
B) `Usavel` não oferece `descrever`.
C) Classe abstrata impede usar métodos concretos.
D) Interface não pode receber objeto concreto.
E) A verificação deveria esperar a execução.

Q13 · RESOLUÇÃO E HABILIDADE

A chamada precisa ser válida antes do despacho

A: letra B. `Usavel` não oferece `descrever`.

- A atribuição a `Usavel` é válida: amuleto cumpre o contrato.
- Métodos concretos herdados continuam utilizáveis pelas subclasses.
- O compilador verifica o tipo declarado antes da execução.

Habilidade: separar compilação e despacho dinâmico.

B: o mesmo argumento está na letra **D**.

CONCEITO-CHAVE

DOS RASTREIOS ÀS DECISÕES DE PROJETO

Duas decisões em momentos diferentes

1. **Compilação:** o tipo declarado oferece a operação?
2. **Execução:** qual implementação o objeto concreto executa?

O despacho não corrige uma chamada rejeitada pelo compilador.

Na próxima etapa, usamos essas distinções para avaliar mudanças.

Q14 · VARIANTE A

Personagem deve herdar de Inventario?

```
public class Personagem extends Inventario { /* ... */ }
```

Requisito: guardar e consultar itens do personagem.

Alternativa	Proposta
A	Precisar dos métodos justifica a herança.
B	Os métodos herdados não acessariam a lista privada.
C	Composição exigiria copiar os métodos.
D	Sobrescrever eliminaria a relação de subtipagem.
E	Ter um inventário permite delegar sem ser um inventário.

Reutilizar também estabelece uma relação

A: letra E. Ter um inventário permite delegar sem ser um inventário.

- Precisar dos métodos não basta para justificar subtipagem.
- Métodos herdados acessam o estado privado definido na base.
- Composição não exige copiar; sobrescrever não elimina subtipagem.

Habilidade: comparar herança e composição pelo significado da relação.

B: o mesmo argumento está na letra **C**.

Q15 · VARIANTE A

O que muda ao receber Equipamento?

```
// Campo: private Usavel equipado;
public void equipar(Equipamento item) {
    equipado = item;
}
```

A) Nada: o parâmetro apenas documenta.
B) O campo `Usavel` deixa de aceitar a referência.
C) Poções continuam aceitas pela assinatura de `usar`.
D) Poções deixam de compilar nas chamadas a `equipar`.
E) O despacho dinâmico deixa de existir.

Q15 · RESOLUÇÃO E HABILIDADE

O parâmetro restringe as chamadas aceitas

A: letra D. Poções deixam de ser aceitas.

`Usavel` já oferece `usar`. `Equipamento` continua compatível com o campo, mas exclui outros implementadores do contrato.

Mesma assinatura não cria subtipagem. A mudança não elimina o despacho.

Habilidade: analisar o impacto de restringir o parâmetro.

B: o argumento correto está na letra **E**.

Q15 · ALCANCE DA RESTRIÇÃO

O tipo do argumento também importa

```
Usavel escolha = new Cajado(1);
personagem.equipar(escolha);
```

Com parâmetro `Usavel`: a chamada é aceita.
Com parâmetro `Equipamento`: é rejeitada pelo tipo declarado do argumento.

Saber que o objeto é um cajado não elimina essa verificação.

Q16 · MUDANÇA LOCAL EM EQUIPAMENTO.RECARREGAR

A proposta substitui o saldo em vez de somar

```
public void recarregar(int quantidade) {
    if (quantidade <= 0) {
        return;
    }
    cargas = quantidade;
    if (cargas > maximo) {
        cargas = maximo;
    }
}
```

ATIVIDADE

Q16 · VARIANTE A

Qual teste passa no original e falha na proposta?

A proposta usa `cargas = quantidade`, mantendo guarda e teto três.

Cada teste cria um cajado novo e verifica `getCargas()`.

Alternativa	Cargas iniciais	Recarga	Expectativa
A	0	1	1
B	2	3	3
C	1	1	2
D	3	5	3
E	2	0	2

Só um teste torna o defeito observável

A: letra C. Uma carga inicial, recarga de uma, expectativa de duas.

Caso	A: 0/1	B: 2/3	C: 1/1	D: 3/5	E: 2/0
Original	1	3	2	3	2
Proposta	1	3	1	3	2

Habilidade: escolher dados que distinguem implementações.

B: esse mesmo teste ocupa a letra **D**.

Q17 · VARIANTE A

Onde proteger a nova regra?

Regra local: alvo com vida zero → `false`, sem efeito nem consumo. Deve valer no uso direto e no uso delegado pelo personagem.

Alternativa	Ponto da guarda
A	Em `Personagem.recuperarVida`, antes de alterar a vida.
B	Em `Equipamento.usar`, antes do efeito e do consumo.
C	Em `Equipamento.usar`, depois do efeito.
D	Somente em `Equipamento.aplicarEm`.
E	Somente em `Personagem.usarEquipadoEm`.

A guarda precisa anteceder todos os efeitos

A: letra B. Verificar em `Equipamento.usar`, antes de `aplicarEm` e do consumo.

- Só no personagem: não cobre a chamada direta ao equipamento.
- Só na recuperação ou no efeito: não controla consumo e retorno.
- Depois de `aplicarEm`: o alvo já pode ter mudado.

Habilidade: proteger a regra no ponto comum aos caminhos exigidos.

B: a mesma posição da guarda está na letra **A**.

Os testes começam com objetos novos

```
// Reiniciar antes de CADA teste:
Cajado item = new Cajado(1);
Personagem alvo = new Personagem(46);

// Teste I
assertEquals(true, item.usar(alvo));

// Teste II
item.usar(alvo);
assertEquals(34, alvo.getVida());
```

Q18 · VARIANTE A

Qual defeito é identificado somente por II?

I: observa o retorno. **II:** observa a vida final, `34`.
Cada teste reinicia cajado com uma carga e alvo com vida `46`.

Alternativa	Retorno	Dano	Consome carga?
A	`true`	nenhum	sim
B	`false`	12	sim
C	`false`	nenhum	não
D	`true`	12	não
E	`false`	24	sim

Retorno e vida são observações diferentes

A: letra A. `true` sem dano passa em I e falha em II.

Alternativa	A	B	C	D	E
Teste I	passa	falha	falha	passa	falha
Teste II	falha	passa	falha	passa	falha

Habilidade: avaliar o alcance e os limites de cada teste.

B: alvo `48`, expectativa `36`; mesmo defeito na letra **E**.

Q18 · LIMITE DA VERIFICAÇÃO

Passar nos dois testes não cobre o consumo

Na alternativa **D**, o método devolve `true` e causa doze de dano, mas não consome carga.

I passa. II também passa. Nenhum observa `getCargas()`.

Um teste só revela um defeito quando a observação final o distingue.

Na Q19, vamos avaliar outra mudança que compila, mas pede julgamento.

Q19 · VARIANTE A

A base agora oferece uma implementação padrão

```
// Em Equipamento: antes abstrato, agora concreto
public int getVariacaoVida() { return 0; }

public class Orbe extends Equipamento {
    public Orbe() { super("Orbe", 2, 4); }
}
```

`Equipamento` continua abstrata. Os demais métodos são mantidos.

Q19 · VARIANTE A

Qual consequência está correta?

A base oferece `getVariacaoVida()` concreto, com retorno zero. `Orbe` só declara seu construtor; `Equipamento` continua abstrata.

Alternativa	Consequência proposta
A	`Orbe` não pode ser instanciada por estender classe abstrata.
B	`Orbe` herda o padrão; cajado e amuleto mantêm sobrescritas.
C	`Orbe` precisa redeclarar todos os métodos das interfaces.
D	`Orbe` precisa redeclarar os campos privados da base.
E	O padrão zero substitui as sobrescritas existentes.

Q19 · RESOLUÇÃO E HABILIDADE

A implementação padrão não apaga sobrescritas

A: letra B. `Orbe` pode ser concreta; cajado e amuleto mantêm suas variações.

- Uma subclasse concreta pode estender uma classe abstrata.
- Métodos herdados podem cumprir interfaces.
- Os métodos da base usam seu estado privado; não duplicam campos.

Habilidade: analisar herança concreta e preservação de sobrescritas.

B: a consequência correta está na letra **D**.

Q19 · CONSEQUÊNCIA DA PROPOSTA

Uma implementação possível ainda precisa de requisito

Com a implementação padrão, `Orbe` usa variação zero.

Se tiver carga, seu uso aceita e consome uma carga sem mudar a vida.

Isso compila. A adequação da regra depende do comportamento desejado.

A Q20 retoma a outra face do problema: proteger uma regra existente.

Q20 · VARIANTE A

O getter permite ultrapassar três itens?

```
public List<Item> getItens() {
    return itens;
}
```

Regra local: `adicionar` já impede inclusão além de três itens.

A) Não: campo privado impede alteração da lista.
B) Não: devolver a lista cria uma cópia.
C) Sim: `add` na lista devolvida contorna `adicionar`.
D) Sim: reatribuir fora substitui o campo privado.
E) Não: seria necessário um setter.

Q20 · RESOLUÇÃO E HABILIDADE

A referência devolvida abre outro caminho de mudança

A: letra C. O cliente pode executar `add` na própria lista interna.

- `private` protege o campo; não torna a lista imutável.
- Devolver a referência não cria cópia nem exige setter para alterar itens.
- Reatribuir a variável externa não substitui o campo do inventário.

Habilidade: diagnosticar exposição de estado mutável.

B: o mesmo diagnóstico está na letra **A**.

SÍNTESE

FECHAMENTO

Três distinções para continuar

- **Referência, objeto e estado:** compartilhar não é copiar; reatribuir não redireciona outras referências.
- **Contrato e execução:** o tipo permite a chamada; o objeto determina a implementação.
- **Regra e verificação:** proteger os caminhos exigidos; observar a propriedade que se quer garantir.

DIAGNÓSTICO INDIVIDUAL

Qual passo do raciocínio você consegue explicar?

Escolha uma questão cujo raciocínio agora consegue explicar e uma habilidade que ainda precisa retomar.

Em cada caso, indique a referência, condição, chamada ou observação que decide o resultado.

Registro de estudo, sem nova nota ou entrega.

CRITÉRIO DE CONFERÊNCIA DO REGISTRO

Um diagnóstico útil identifica a ação

Já consigo explicar Q5: trocar `escolha` não altera o campo `equipado`; o cajado continua recebendo a chamada.

Preciso retomar Q12: seguir a delegação até a base e aplicar as curas somente na volta com `true`.

O roteiro reúne o código completo, as resoluções expansíveis e os dois gabaritos.